

Chapitre N°5:

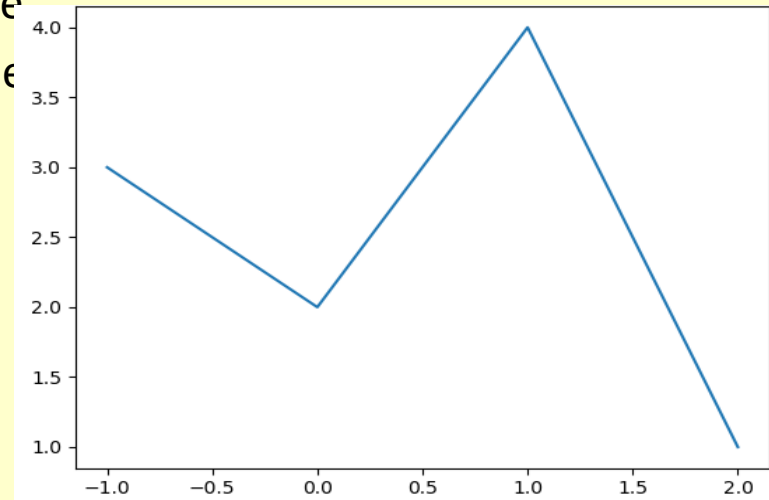
Visualisation de données avec Matplotlib et Seaborn

MATPLOTLIB

- Matplotlib est une bibliothèque de Python destinée à tracer et visualiser des données sous formes de graphiques. Combinée avec numpy, elle permet une analyse des simulations numériques, à la fois sous forme interactive et pour produire des documents de très grande qualité, directement utilisable dans les publications scientifiques.
- Il faut importer la librairie matplotlib et son module pyplot ainsi que numpy:
- **import numpy as np**
- **import matplotlib.pyplot as plt**
- Le principe est simple : on crée une liste d'abscisses, une liste d'ordonnées et on demande de tracer les points de coordonnées correspondantes.
- Par défaut, les points tracés sont reliés par des segments de droite, ce qui donne une courbe. On peut demander aussi à ce que les points ne soient pas reliés, on obtient un nuage de points.

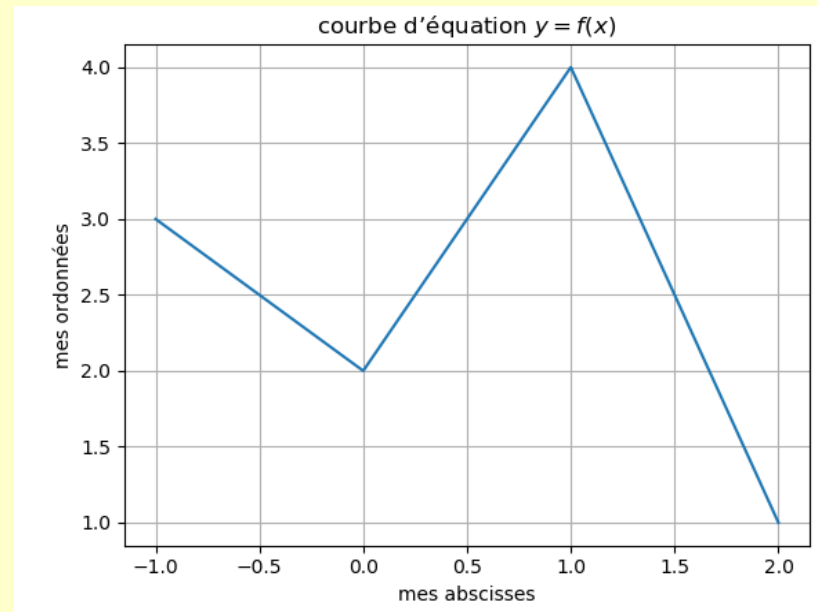
MATPLOTLIB

- Par exemple, en exécutant le script suivant, on obtient un premier graphique.
 - `import matplotlib.pyplot as plt`
 - `X = [-1, 0, 1, 2]` # la liste des abscisses
 - `Y = [3, 2, 4, 1]` # la liste des ordonnées
 - `plt.plot(X, Y)` # pour créer le graphique
 - `plt.show()` # pour afficher le graphique



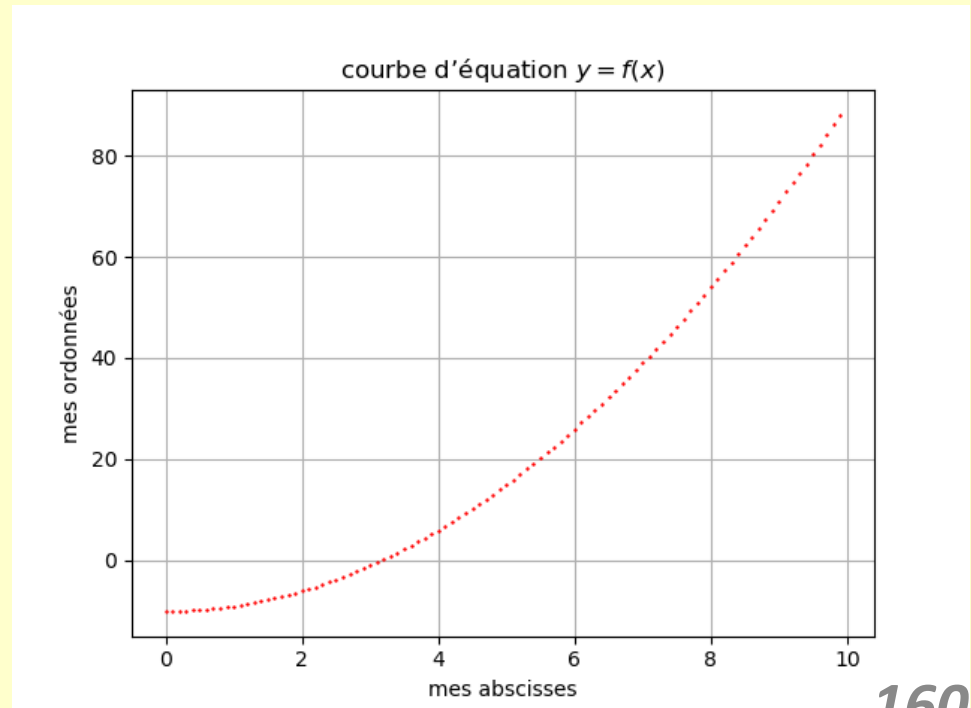
MATPLOTLIB

- Si l'on insère les commandes suivantes, on complète le premier graphique.
 - `plt.grid()` # pour créer une grille
 - `plt.xlabel("mes abscisses")`
 - `plt.ylabel("mes ordonnées")`
 - `plt.title("courbe d'équation  $y = f(x)$ ")`



Exemple :

- `import matplotlib.pyplot as plt`
- `import numpy as np`
- `x = np.arange(0,10,0.1)`
- `y = x**2-10`
- `plt.plot(x, y, color = 'red', linestyle = 'none', linewidth = 1, marker = '+', markerfacecolor = 'none', markersize = 1) # pour créer le graphique`
- `plt.grid() # pour créer une grille`
- `plt.xlabel("mes abscisses")`
- `plt.ylabel("mes ordonnées")`
- `plt.title("courbe d'équation  $y = f(x)$ ")`
- `plt.show() # pour afficher le graphique`



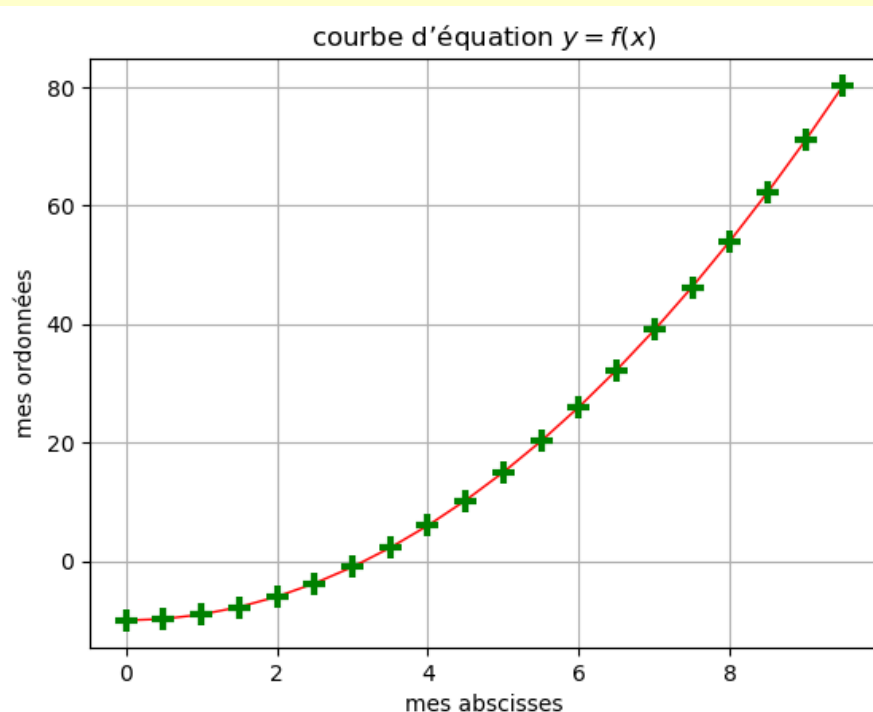
Exemple :

- `color = 'red'` : couleur du trait.
- `linewidth = 5` : largeur du trait.
- `linestyle = 'dotted'` : style du trait (valeurs possibles : 'solid', 'dashed', 'dashdot', 'dotted').
- `solid_capstyle = 'butt'` : bout carré qui se termine exactement à la position souhaitée.
- `marker = 'D'` : indique de mettre un symbole aux extrémités de la ligne.
- `markersize = 15` : taille du marqueur s'il y en a un.
- `markerfacecolor = 'yellow'` : la couleur du marqueur (l'intérieur).
- `markeredgecolor = 'cyan'` : la couleur des bordures du marqueur.
- `markeredgewidth = 3` : l'épaisseur des bordures du marqueur.

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 x = np.arange(0,10,0.5)
4 y = x**2-10
5 plt.plot(x, y, color = 'red', linestyle = 'solid', linewidth = 1,
6         marker = '+', markeredgecolor = 'green', markersize = 10,
7         markeredgewidth = 3) |
8 plt.grid() # pour créer une grille
9 plt.xlabel("mes abscisses")
10 plt.ylabel("mes ordonnées")
11 plt.title("courbe d'équation $y =f(x)$")
12 plt.show() # pour afficher le graphique

```



Exemple :

- Comment tracer une courbe d'une fonction?

```
import matplotlib.pyplot as plt

x = []
y = [] # On a créé deux Listes vides pour contenir Les abscisses et Les ordonnées

pas = 0.1 # On définit un pas de 0.1 pour une meilleure précision
abscisse = -2 # L'abscisse de départ est -2

for i in range(41): # On parcourt Les valeurs de x de -2 à 2
    x.append(abscisse)
    y.append(abscisse**2) # On rajoute L'abscisse et son image
    abscisse = abscisse+pas # on augmente abscisse de pas pour passer au point suivant

plt.plot(x, y)
plt.show()
```

Exemple :

```
import matplotlib.pyplot as plt
from math import cos, pi

x = []
y = [] # On a créé deux Listes vides pour contenir Les abscisses et Les ordonnées

pas = 2 * pi / 100
abscisse = 0 # L'abscisse de départ est 0

for i in range(0, 101): # On veut Les points de 0 à 100
    x.append(abscisse)
    y.append(cos(abscisse)) # On rajoute l'abscisse et son image par la fonction cos
    abscisse += pas # on augmente abscisse de pas pour passer au point suivant

plt.plot(x, y)
plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0, 2 * np.pi, 0.01) # On crée un array qui va de 0 à 2pi exclu avec un pas de 0.01
plt.plot(x, np.cos(x)) # On utilise plot avec l'array x et l'array cos(x)
plt.show()
```

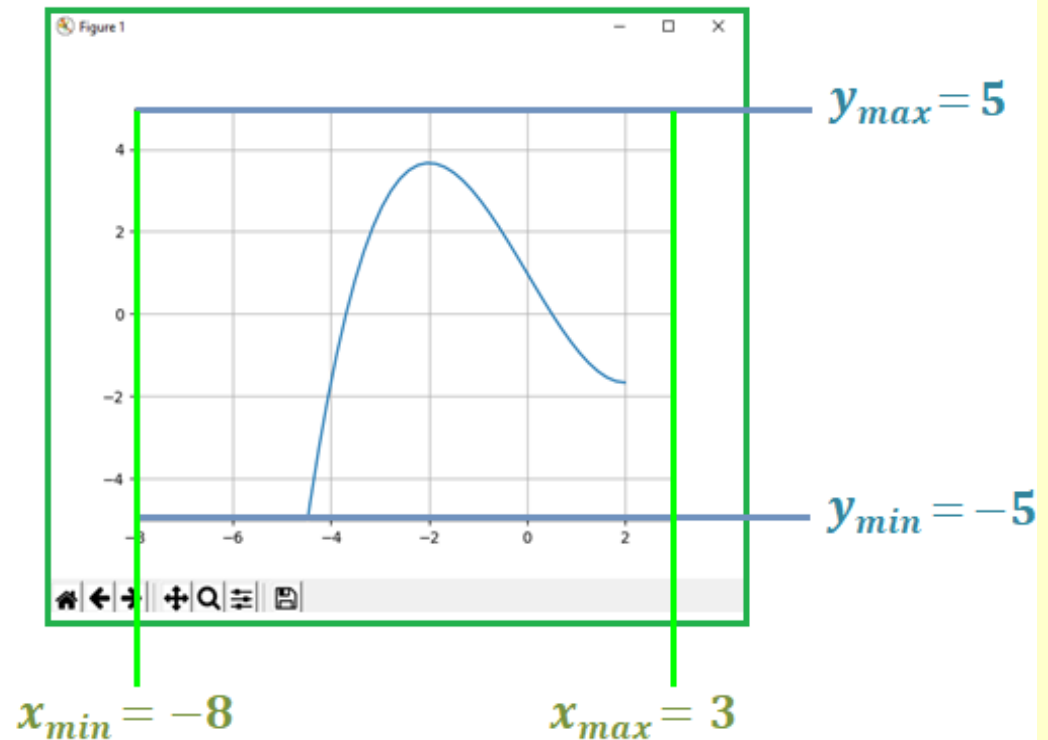
Exemple :

```
plt.axis([ $x_{min}$ ,  $x_{max}$ ,  $y_{min}$ ,  $y_{max}$ ])
```

Cette instruction n'est pas obligatoire. On l'utilise si l'on veut imposer la taille de la fenêtre d'affichage.

exemple

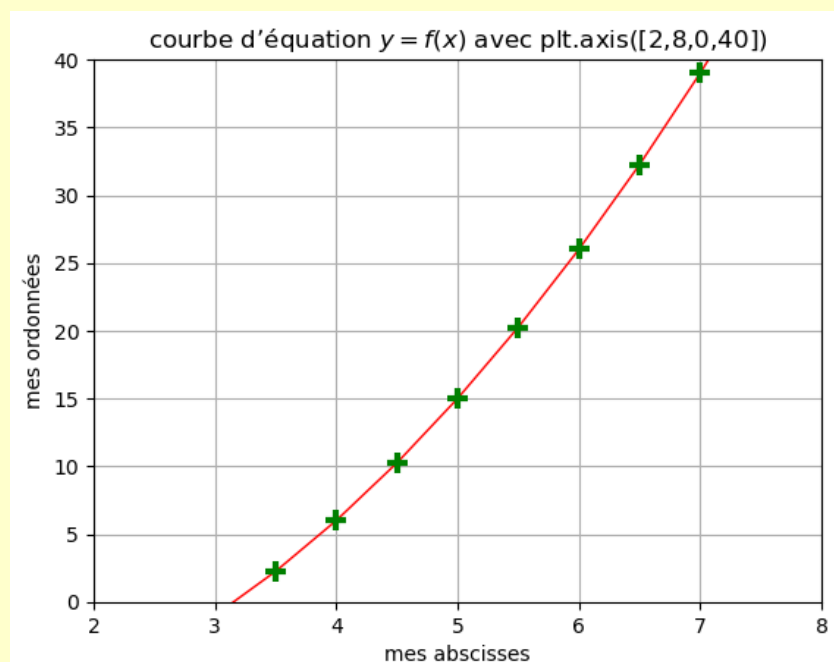
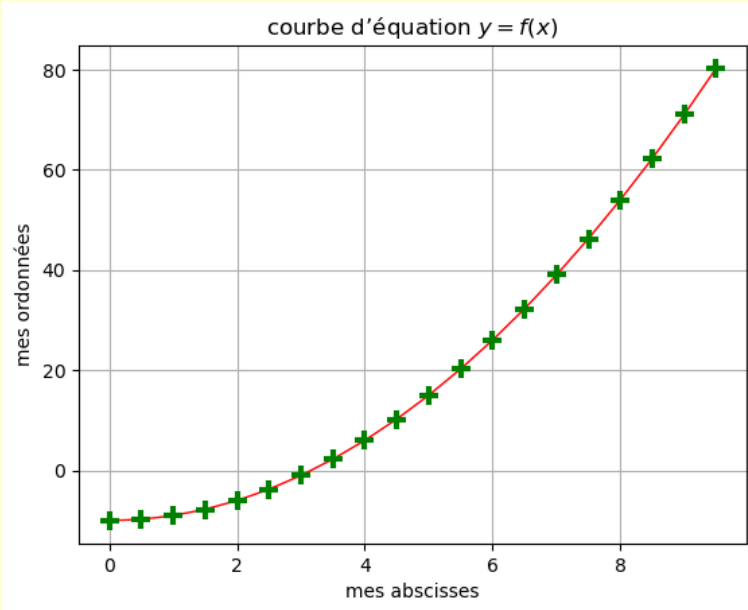
```
import matplotlib.pyplot as plt
import numpy as np
plt.axis([-8,3,-5,5])
plt.grid()
x=np.linspace(-7,2,50)
y=x**3/6-2*x+1
plt.plot(x,y)
plt.show()
```



```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 x = np.arange(0,10,0.5)
4 y = x**2-10
5 #avec plt.axis().....
6 plt.axis([2,8,0,40])
7 plt.plot(x, y, color = 'red', linestyle = 'solid', linewidth = 1,
8         marker = '+', markeredgecolor = 'green', markersize = 10,
9         markeredgewidth = 3)
10 plt.grid() # pour créer une grille
11 plt.xlabel("mes abscisses")
12 plt.ylabel("mes ordonnées")
13 plt.title("courbe d'équation $y =f(x)$ avec plt.axis([2,8,0,40])")
14 plt.show() # pour afficher le graphique

```



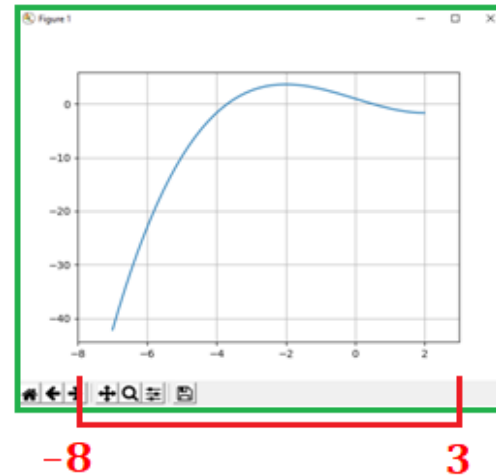
Exemple :

`plt.xlim( $x_{min}$ ,  $x_{max}$ )` et `plt.ylim( $y_{min}$ ,  $y_{max}$ )`

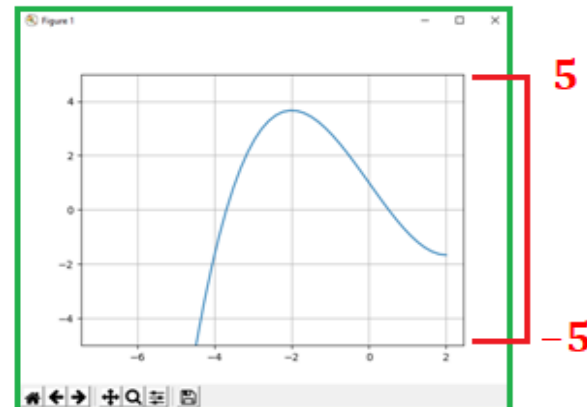
Il est aussi possible de fixer uniquement l'axe des abscisses ou uniquement l'axe des ordonnées avec « *xlim* » et « *ylim* ».

exemple

```
import matplotlib.pyplot as plt
import numpy as np
plt.xlim(-8,3)
plt.grid()
x=np.linspace(-7,2,50)
y=x**3/6-2*x+1
plt.plot(x,y)
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np
plt.ylim(-5,5)
plt.grid()
x=np.linspace(-7,2,50)
y=x**3/6-2*x+1
plt.plot(x,y)
plt.show()
```



Exemple :

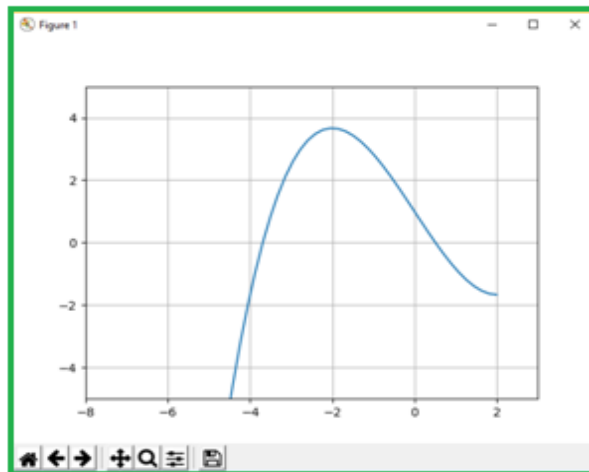
`plt.grid()`

Cette instruction fait afficher un quadrillage.

exemple

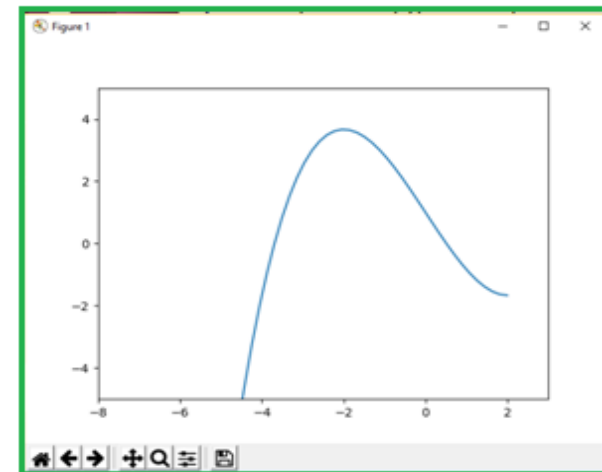
avec « grid »

```
import matplotlib.pyplot as plt
import numpy as np
plt.axis([-8,3,-5,5])
plt.grid()
x=np.linspace(-7,2,50)
y=x**3/6-2*x+1
plt.plot(x,y)
plt.show()
```



sans « grid »

```
import matplotlib.pyplot as plt
import numpy as np
plt.axis([-8,3,-5,5])
x=np.linspace(-7,2,50)
y=x**3/6-2*x+1
plt.plot(x,y)
plt.show()
```



Exemple :

Couleur et style des courbes

On peut changer le style des points ou de la courbe en ajoutant une chaîne de caractère dans `plot` :

```
plt.plot(x,y,"bs--")
```

ici la courbe sera bleue, les points de forme carrée, et la ligne en traits pointillés

style du trait	syntaxe
—	"_"
---	"_ _"
.....	".."
--..--	"_ ."

← style du trait par défaut

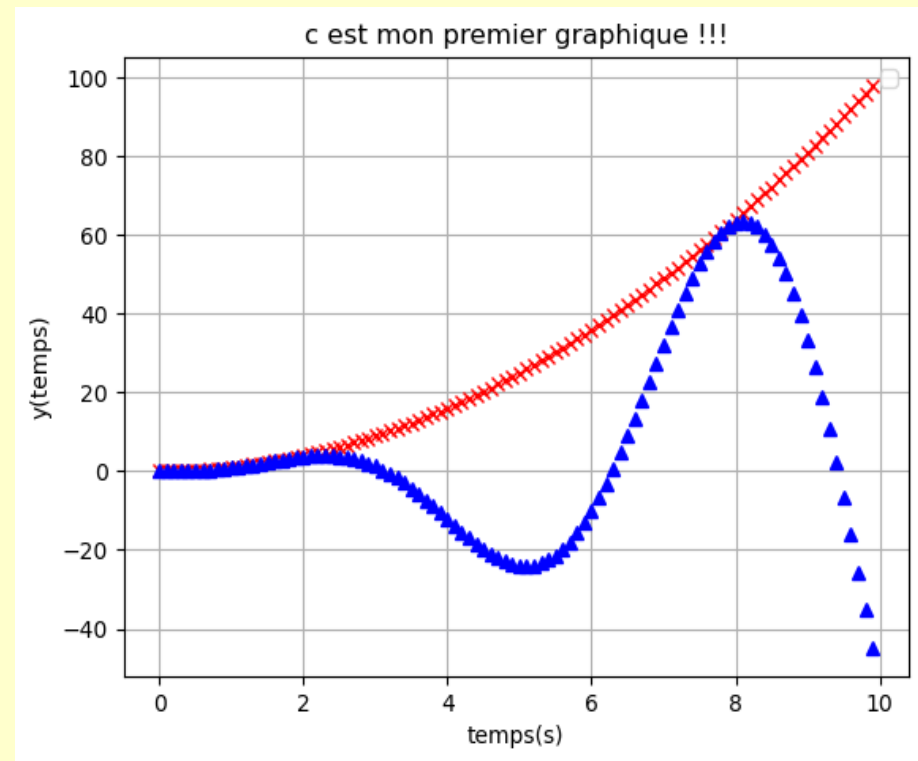
couleur	syntaxe
	"b" ou "blue"
	"c" ou "cyan"
	"g" ou "green"
	"k" ou "black"
	"m" ou "magenta"
	"r" ou "red"
	"y" ou "yellow"

style du point	syntaxe
+	"+"
×	"x"
●	"o"
■	"s"
★	"*"
▲	"^"
▼	"v"
►	">"
◄	"<"

```

1 from matplotlib.pyplot import *
2 from numpy import *
3
4 def g(x):
5     return sin(x)*x**2
6
7 def f(x):
8     return x**2
9 t=arange(0,10,0.1)
10 Y1=f(t)
11 Y2=g(t)
12 xlabel("temps(s)")
13 ylabel("y(temps)")
14 title("c est mon premier graphique !!!")
15 legend(('x->x**2', 'x->sin(x)*x**2'))
16 plot(t,Y1,'rx')
17 plot(t,Y2,'b^')
18 grid()
19 show()

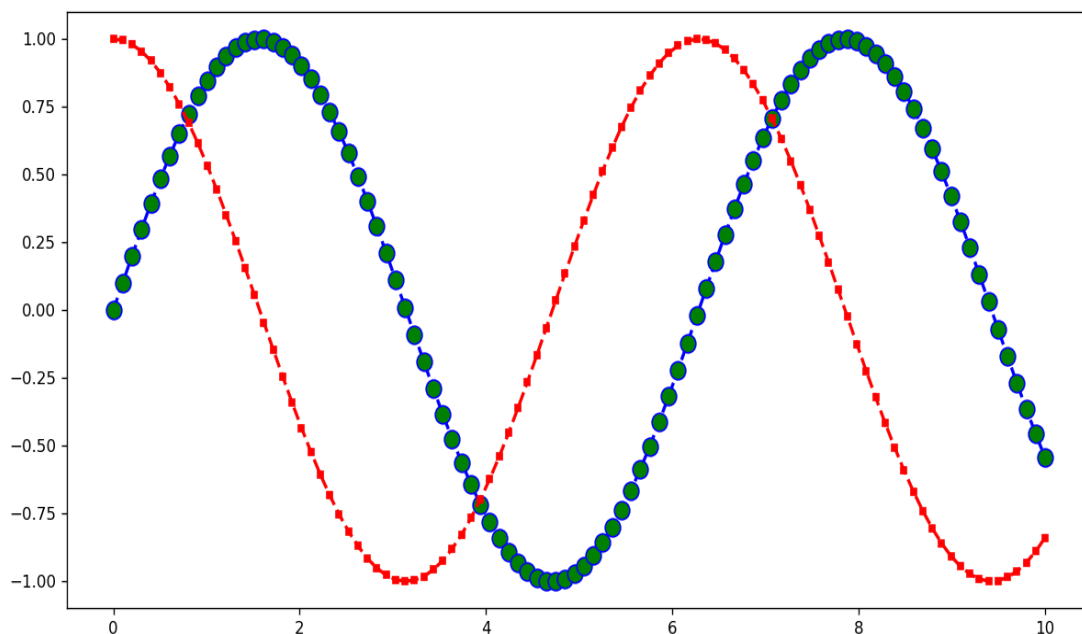
```



```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 # --- Données ---
4 x = np.linspace(0, 10, 100)
5 y1 = np.sin(x) ; y2 = np.cos(x)
6 # --- Création de la figure ---
7 plt.figure(figsize=(12, 6), dpi=120, facecolor='white')
8 # --- Courbes ---
9 plt.plot(x, y1, label='sin(x)', color='blue', linestyle='--', linewidth=2,
10          marker='o', markersize=10, markerfacecolor='g')
11 plt.plot(x, y2, label='cos(x)', color='red', linestyle='--', linewidth=2,
12          marker='s', markersize=4)
13 plt.show()

```



dpi dans `plt.figure()` signifie **dots per inch** : le nombre de points (pixels) affichés par pouce.

En pratique :

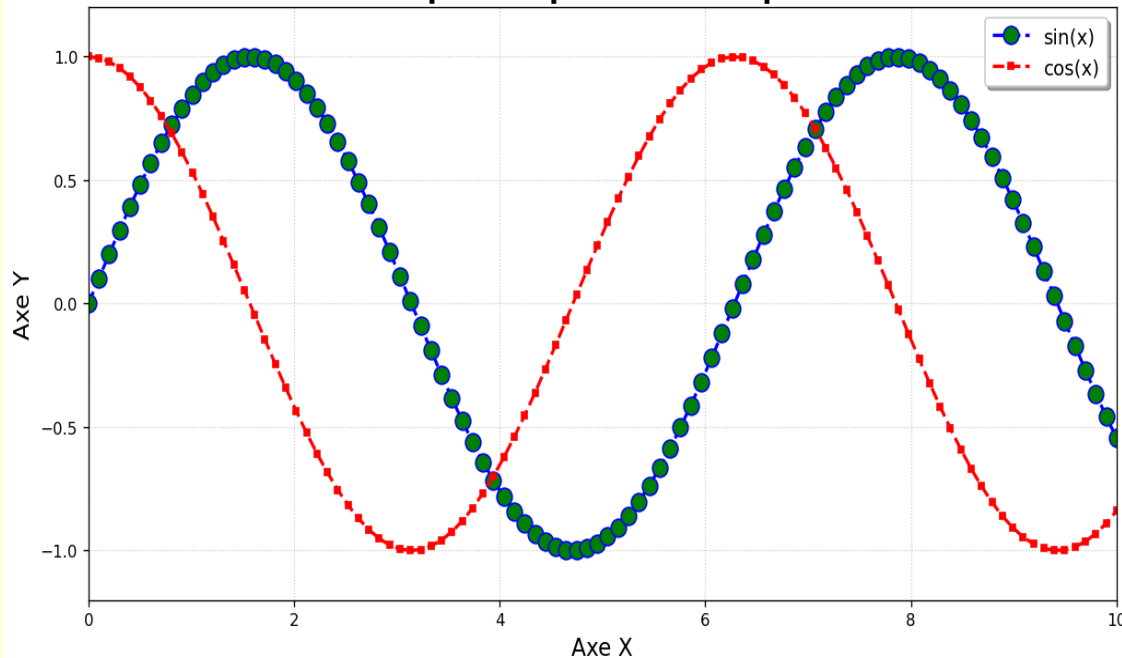
- **dpi plus élevé** → **image plus nette**, plus dense en pixels.
- **dpi plus faible** → **image plus pixellisée**, mais plus légère à générer.

```

7 ##### Nouvelle Partie 1 #####
8 plt.title("Exemple complet avec matplotlib", fontsize=18, fontweight='bold')
9 plt.xlabel("Axe X", fontsize=14) ; plt.ylabel("Axe Y", fontsize=14)
10 # --- Grille ---
11 plt.grid(True, which='both', linestyle=':', linewidth=0.7, alpha=0.7)
12 # --- Légende ---
13 plt.legend(loc='upper right', fontsize=12, frameon=True, shadow=True)
14 # --- Limites des axes ---
15 plt.xlim(0, 10) ; plt.ylim(-1.2, 1.2)
16 # --- Affichage ---
17 plt.show()

```

Exemple complet avec matplotlib



which='both' indique :
graduations majeures (major ticks) et
graduations mineures (minor ticks)

Donc :

which='major' → uniquement les lignes principales

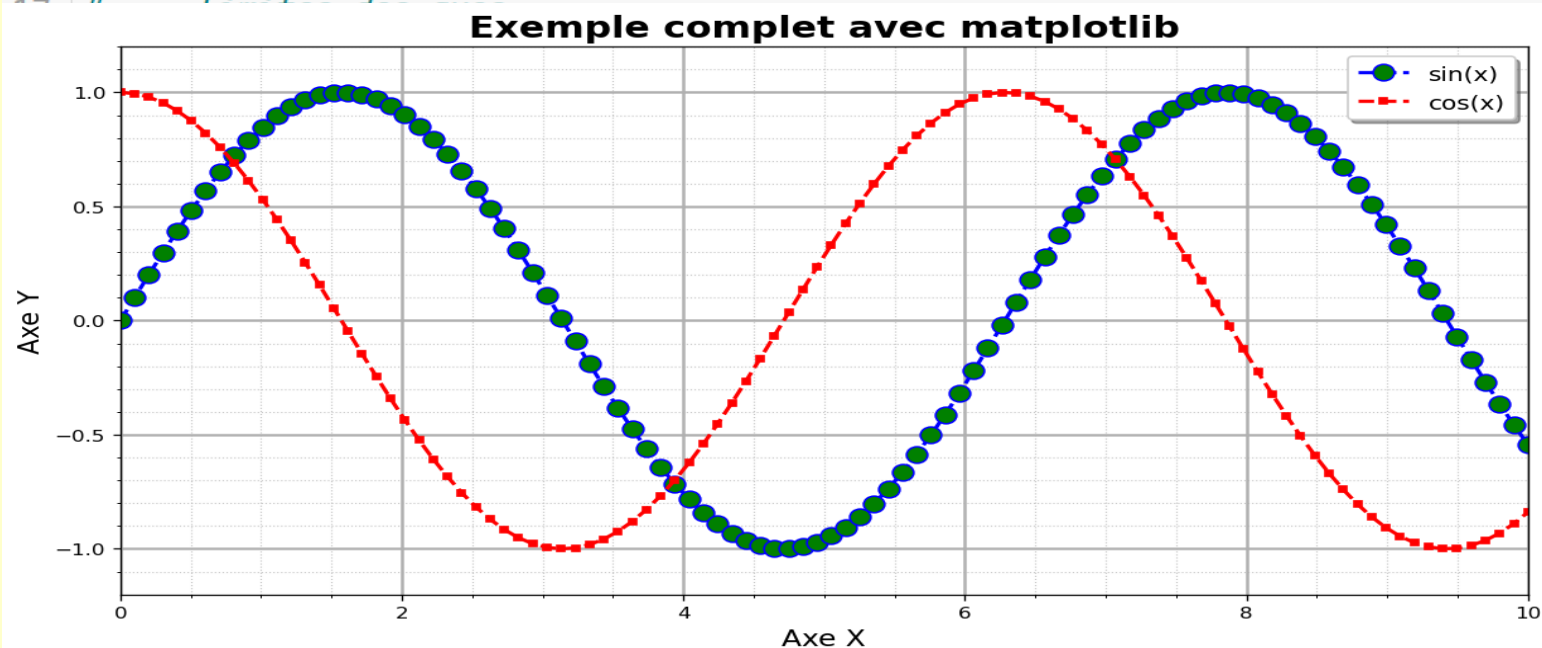
which='minor' → uniquement les petites subdivisions

which='both' → les deux à la fois

```

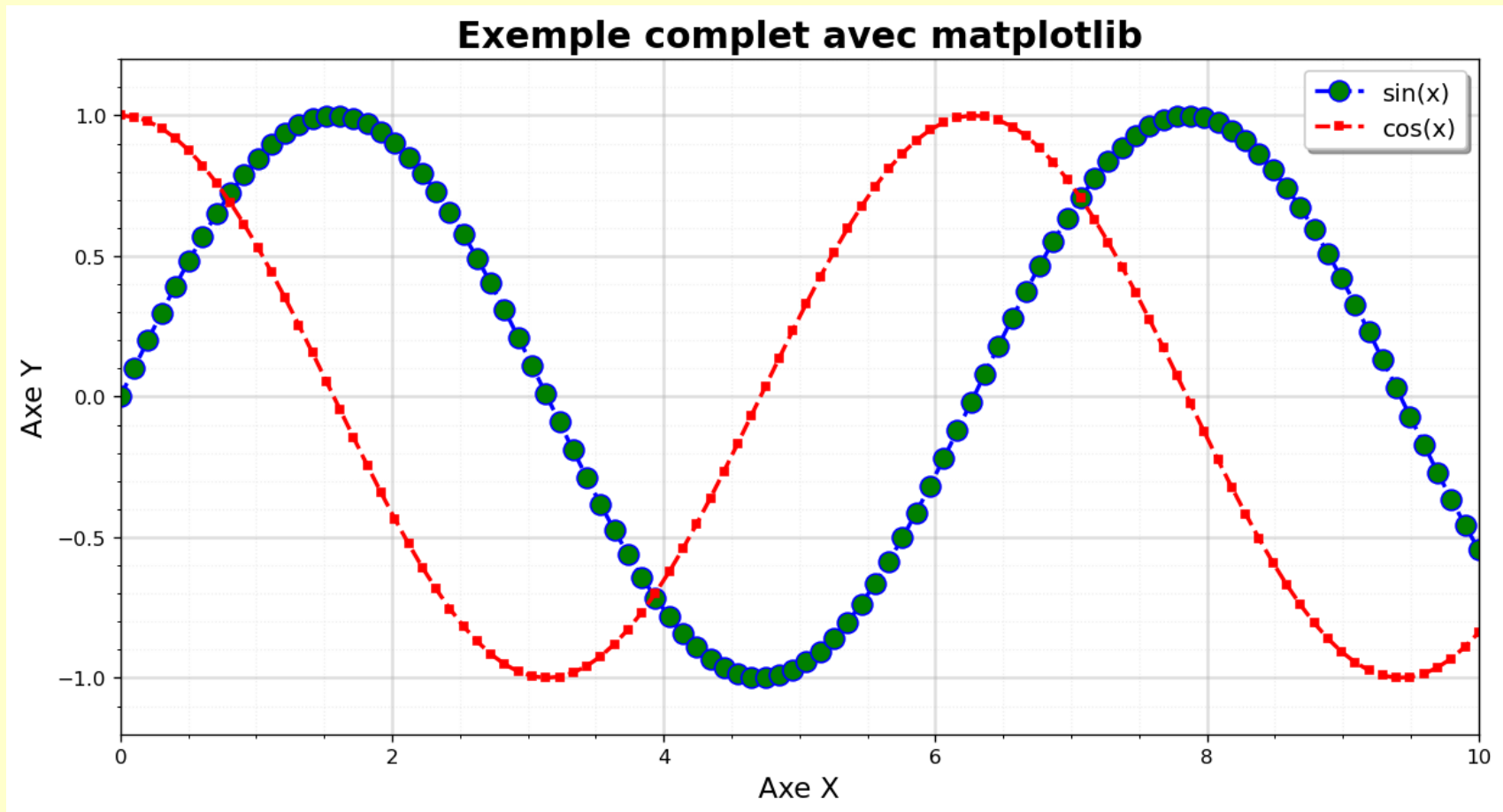
1 plt.figure(figsize=(12, 6), dpi=120, facecolor='white')
2 # --- Courbes ---
3 plt.plot(x, y1, label='sin(x)', color='blue', linestyle='--', linewidth=2,
4          marker='o', markersize=10, markerfacecolor='g')
5 plt.plot(x, y2, label='cos(x)', color='red', linestyle='--', linewidth=2,
6          marker='s', markersize=4)# --- Titre et Labels ---
7 ##### Nouvelle Partie 1 #####
8 plt.title("Exemple complet avec matplotlib", fontsize=18, fontweight='bold')
9 plt.xlabel("Axe X", fontsize=14) ; plt.ylabel("Axe Y", fontsize=14)
10 # --- Grille ---
11 plt.minorticks_on()
12 #plt.grid(True, which='both', linestyle=':', linewidth=0.7, alpha=0.7)
13 plt.grid(True, which='major', linestyle='-', linewidth=1.5, alpha=1)
14 plt.grid(True, which='minor', linestyle=':', linewidth=0.7, alpha=0.7)
15 # --- Légende ---
16 plt.legend(loc='upper right', fontsize=12, frameon=True, shadow=True)

```



```
11 plt.minorticks_on()
12 #plt.grid(True, which='both', linestyle=':', linewidth=0.7, alpha=0.7)
13 plt.grid(True, which='major', linestyle='-', linewidth=1.5, alpha=0.4)
14 plt.grid(True, which='minor', linestyle=':', linewidth=0.7, alpha=0.2)
```

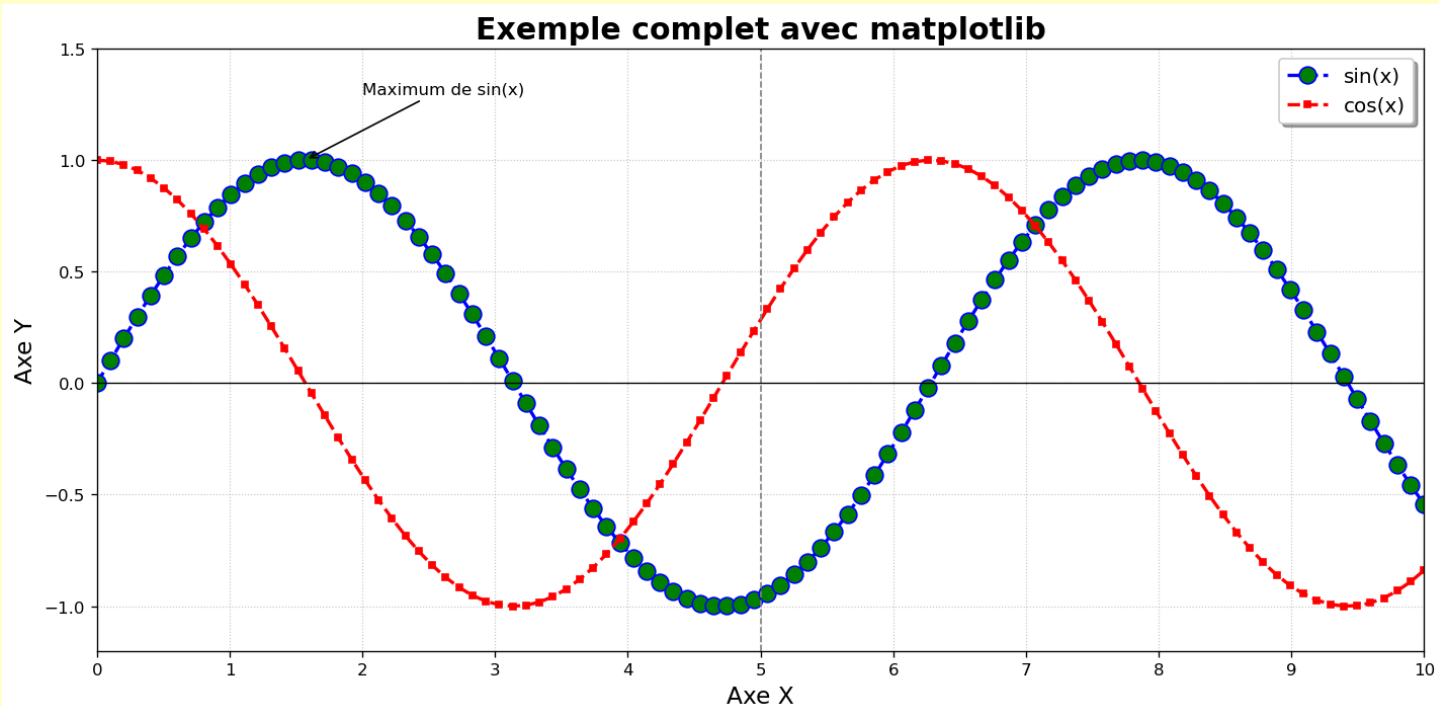
alpha: Transparence de la grille (0 = transparent, 1 = opaque).



```

17 ##### Nouvelle Partie 2 #####
18 # --- Graduation personnalisée ---
19 plt.xticks(ticks=np.arange(0, 11, 1))
20 plt.yticks(ticks=np.arange(-1, 2, 0.5))
21 # --- Ajout d'une annotation ---
22 plt.annotate("Maximum de sin(x)",
23             xy=(np.pi/2, 1),
24             xytext=(2, 1.3),
25             arrowprops=dict(arrowstyle="->", color="black"))
26 # --- Ligne verticale et horizontale ---
27 plt.axvline(x=5, color='gray', linestyle='--', linewidth=1)
28 plt.axhline(y=0, color='black', linewidth=0.8)
29 # --- Ajustement des marges ---
30 plt.tight_layout()
31 # --- Affichage ---
32 plt.show()
33 # --- Sauvegarde ---
34 plt.savefig("exemple_matplotlib.png", dpi=300)

```



Matplotlib vs Seaborn

- **Matplotlib** = bibliothèque de base, très complète mais bas-niveau.
- **Seaborn** = extension qui simplifie, embellit et automatise les graphes statistiques.

Seaborn : couche au-dessus, plus simple et plus beau

✓ Seaborn est basé sur Matplotlib

- Il ne remplace PAS Matplotlib.
- Il facilite son usage.

✓ Niveau haut : automatise beaucoup de choses

Couleurs, styles, échelles, légendes, calculs statistiques.

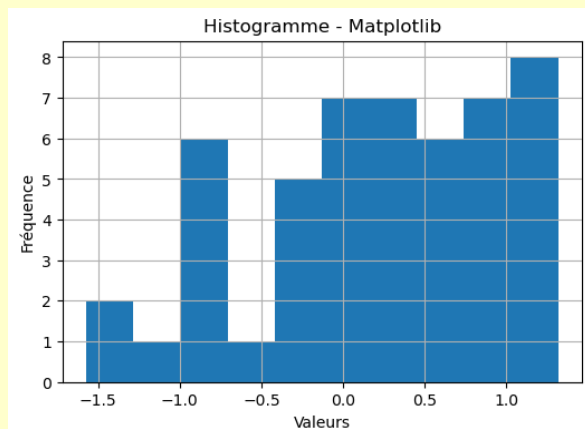
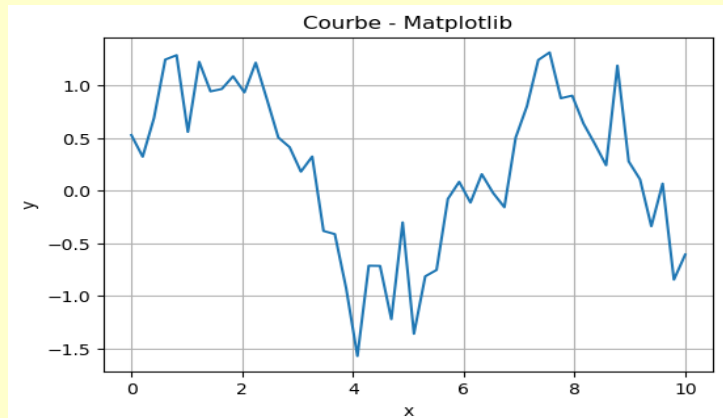
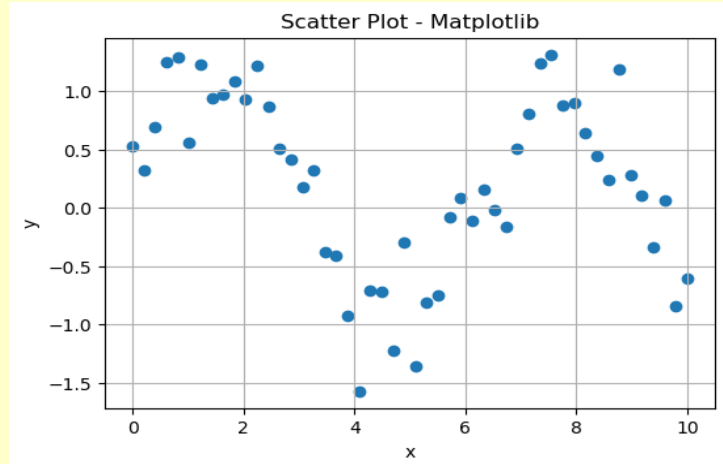
Matplotlib vs Seaborn

Critère	Matplotlib	Seaborn
Niveau	Bas	Haut
Style par défaut	Simple	Esthétique
Statistiques intégrées	✗ Non	✓ Oui
Facilité d'utilisation	6/10	9/10
Personnalisation	✓ Très élevée	Moyenne
Objectif	Contrôle total	Productivité + jolis graphiques
Utilise l'autre ?	Non	✓ Utilise Matplotlib

```

1 import numpy as np ; import pandas as pd
2 import matplotlib.pyplot as plt ; import seaborn as sns
3 np.random.seed(0)
4 x = np.linspace(0, 10, 50)
5 y = np.sin(x) + np.random.normal(scale=0.3, size=50)
6 df = pd.DataFrame({ "x": x, "y": y})
7 # PARTIE 1 - MATPLOTLIB
8 # =====
9 # --- Scatter Matplotlib ---
10 plt.figure(figsize=(6, 4))
11 plt.scatter(x, y)
12 plt.title("Scatter Plot - Matplotlib")
13 plt.xlabel("x") ; plt.ylabel("y")
14 plt.grid(True) ; plt.show()
15 # --- Courbe Matplotlib ---
16 plt.figure(figsize=(6, 4)); plt.plot(x, y)
17 plt.title("Courbe - Matplotlib")
18 plt.xlabel("x"); plt.ylabel("y")
19 plt.grid(True) ; plt.show()
20 # --- Histogramme Matplotlib ---
21 plt.figure(figsize=(6, 4)); plt.hist(y, bins=10)
22 plt.title("Histogramme - Matplotlib")
23 plt.xlabel("Valeurs") ; plt.ylabel("Fréquence")
24 plt.grid(True); plt.show()

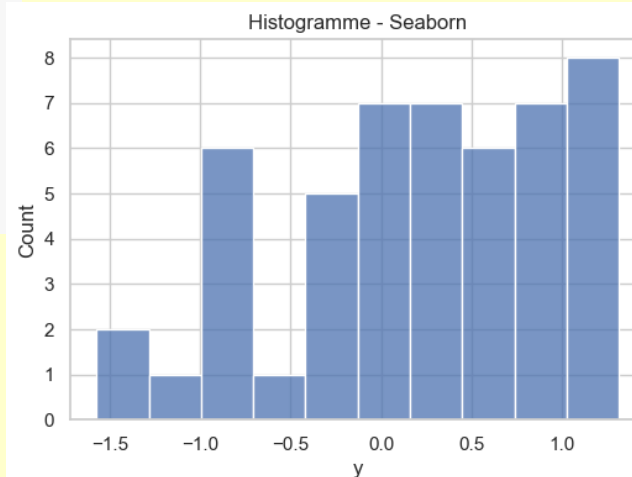
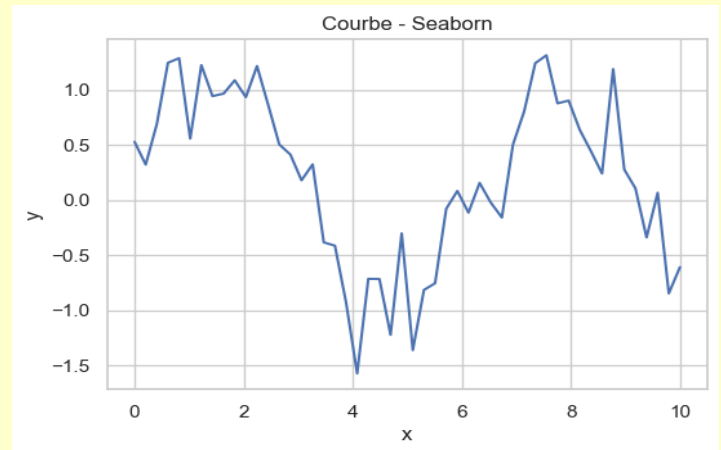
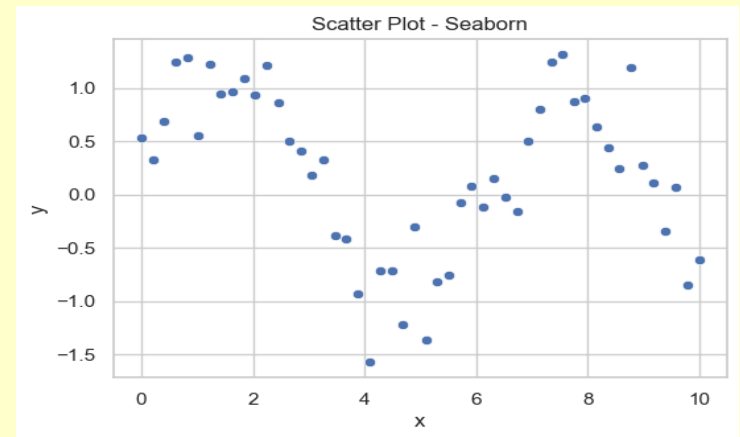
```



```

26 # =====
27 #  PARTIE 2 – SEABORN
28 # =====
29 print("=== Visualisations Seaborn ===")
30 # activation d'un style propre
31 sns.set_theme(style="whitegrid")
32 # --- Scatter Seaborn ---
33 plt.figure(figsize=(6, 4))
34 sns.scatterplot(data=df, x="x", y="y")
35 plt.title("Scatter Plot - Seaborn")
36 plt.show()
37 # --- Courbe Seaborn ---
38 plt.figure(figsize=(6, 4))
39 sns.lineplot(data=df, x="x", y="y")
40 plt.title("Courbe - Seaborn")
41 plt.show()
42 # --- Histogramme Seaborn ---
43 plt.figure(figsize=(6, 4))
44 sns.histplot(data=df, x="y", bins=10)
45 plt.title("Histogramme - Seaborn")
46 plt.show()

```



Styles disponibles dans `sns.set_theme(style=...)`

1. `"darkgrid"` :

Fond gris + grille claire → idéal pour la plupart des visualisations.

2. `"whitegrid"`

Fond blanc + grille → très utilisé en statistiques.

3. `"dark"`

Fond gris foncé, **sans** grille → visuel plus sobre.

4. `"white"`

Fond blanc, **sans** grille → minimaliste et épuré.

5. `"ticks"`

Fond blanc avec axes marqués (ticks accentués) → style scientifique.

Pour ajouter des styles personnalisés:

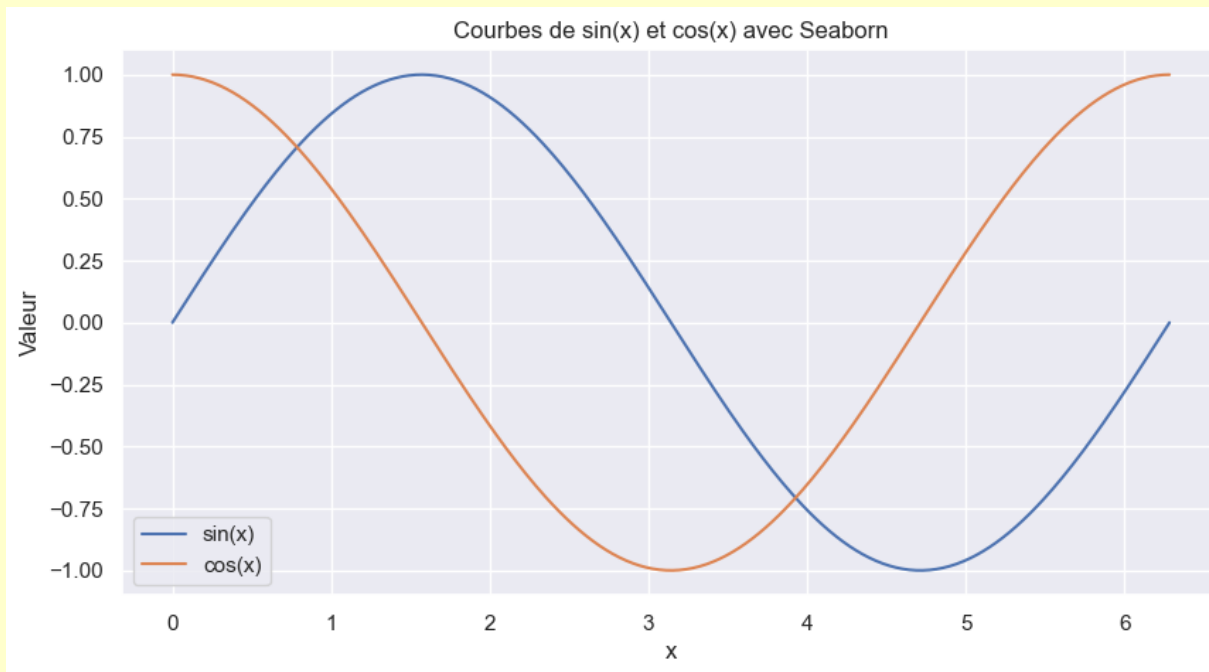
Utiliser `rc + set_style()`

voir la diapositive après la suivante

```

1 import numpy as np, pandas as pd
2 import seaborn as sns, matplotlib.pyplot as plt
3 # Génération des données
4 x = np.linspace(0, 2 * np.pi, 300)
5 data = pd.DataFrame({
6     "x": x, "sin(x)": np.sin(x), "cos(x)": np.cos(x)})
7 # Configuration du style Seaborn
8 sns.set_theme(style="darkgrid")
9 # Tracé
10 plt.figure(figsize=(10, 5))
11 sns.lineplot(data=data, x="x", y="sin(x)", label="sin(x)")
12 sns.lineplot(data=data, x="x", y="cos(x)", label="cos(x)")
13 plt.title("Courbes de sin(x) et cos(x) avec Seaborn")
14 plt.xlabel("x") ; plt.ylabel("Valeur")
15 plt.legend(); plt.show()

```



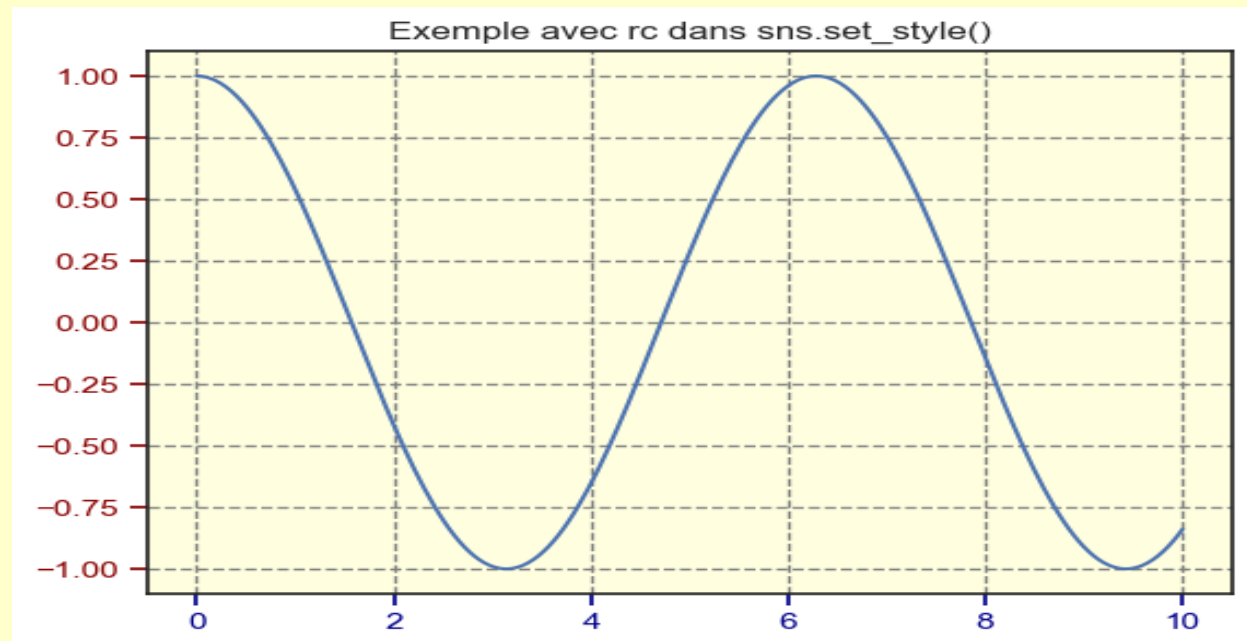
```

1 import seaborn as sns, matplotlib.pyplot as plt
2 import numpy as np
3 sns.set_style("ticks", rc={
4     "axes.facecolor": "lightyellow",      # couleur du fond des axes
5     "grid.color": "gray",                # couleur de la grille
6     "grid.linestyle": "--",              # style de la grille
7     "grid.linewidth": 0.4,               # épaisseur
8     "xtick.color": "darkblue",           # couleur des ticks
9     "ytick.color": "darkred",            # couleur des ticks
10    "figure.figsize": (8, 4) })           # taille de la figure
11 # Données
12 x = np.linspace(0, 10, 200); y = np.cos(x)
13 plt.plot(x, y)
14 plt.title("Exemple avec rc dans sns.set_style()")
15 plt.grid(True) ; plt.show()

```

Pour ajouter des styles personnalisés:

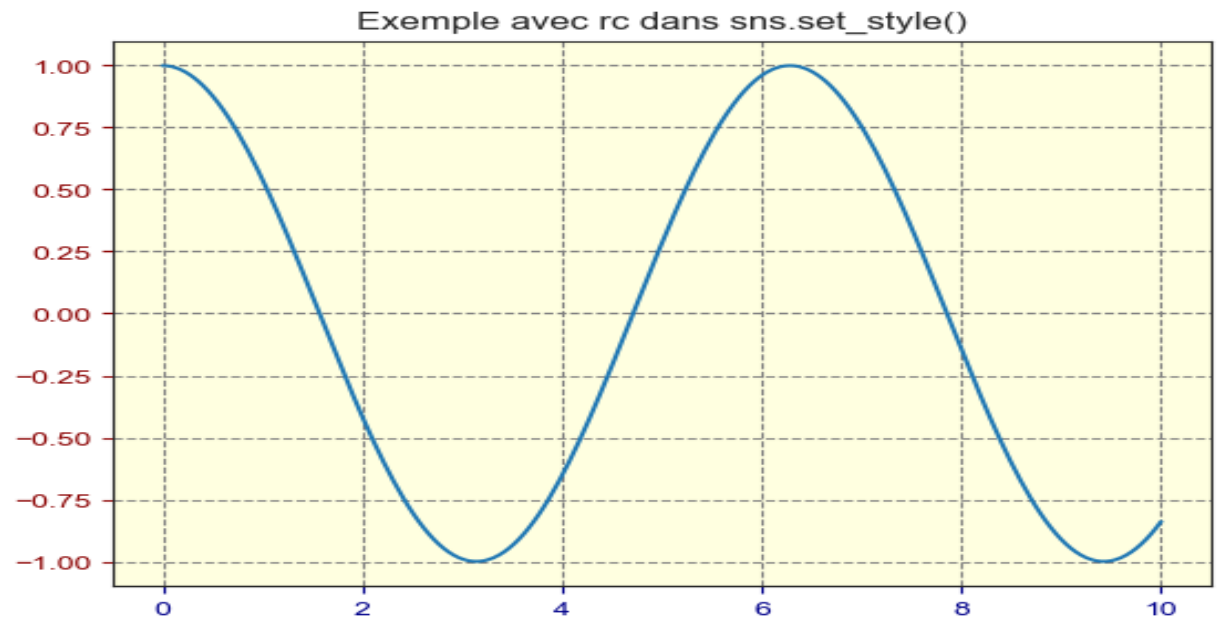
rc + set_style()



```

1 import seaborn as sns
2 import numpy as np
3 # Style
4 sns.set_style("ticks", rc={"axes.facecolor": "lightyellow",
5     "grid.color": "gray",      "grid.linestyle": "--",
6     "grid.linewidth": 0.4,    "xtick.color": "darkblue",
7     "ytick.color": "darkred", "figure.figsize": (8, 4) })
8
9 # Données
10 x = np.linspace(0, 10, 200); y = np.cos(x)
11 # Tracé avec seaborn seulement
12 ax = sns.lineplot(x=x, y=y)
13 ax.set_title("Exemple avec rc dans sns.set_style()")
14 # Activer la grille (on n'utilise pas pyplot ici)
15 ax.grid(True)
16 # Affichage (Seaborn appelle Matplotlib automatiquement)
17 ax.figure.show()

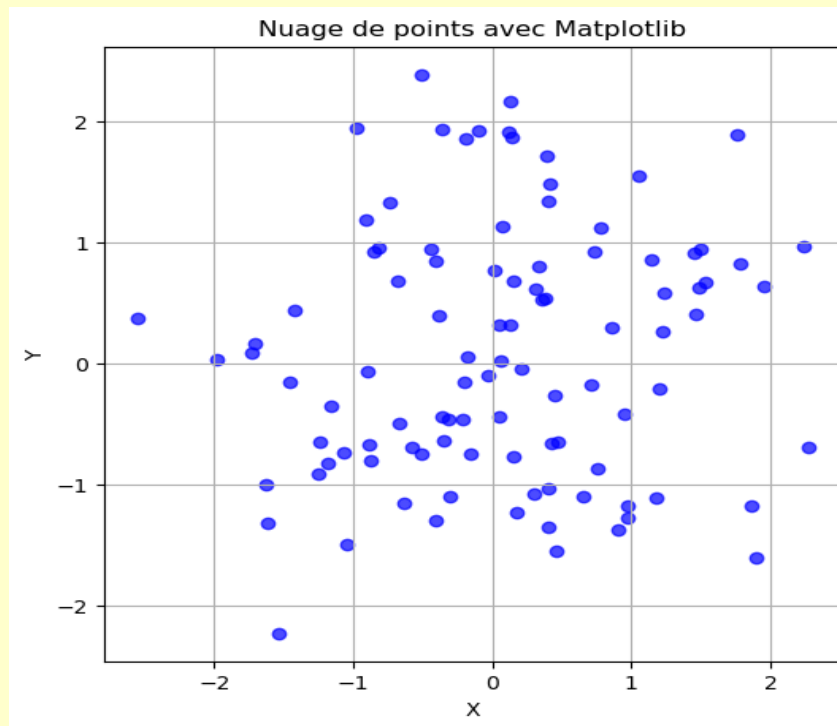
```



```

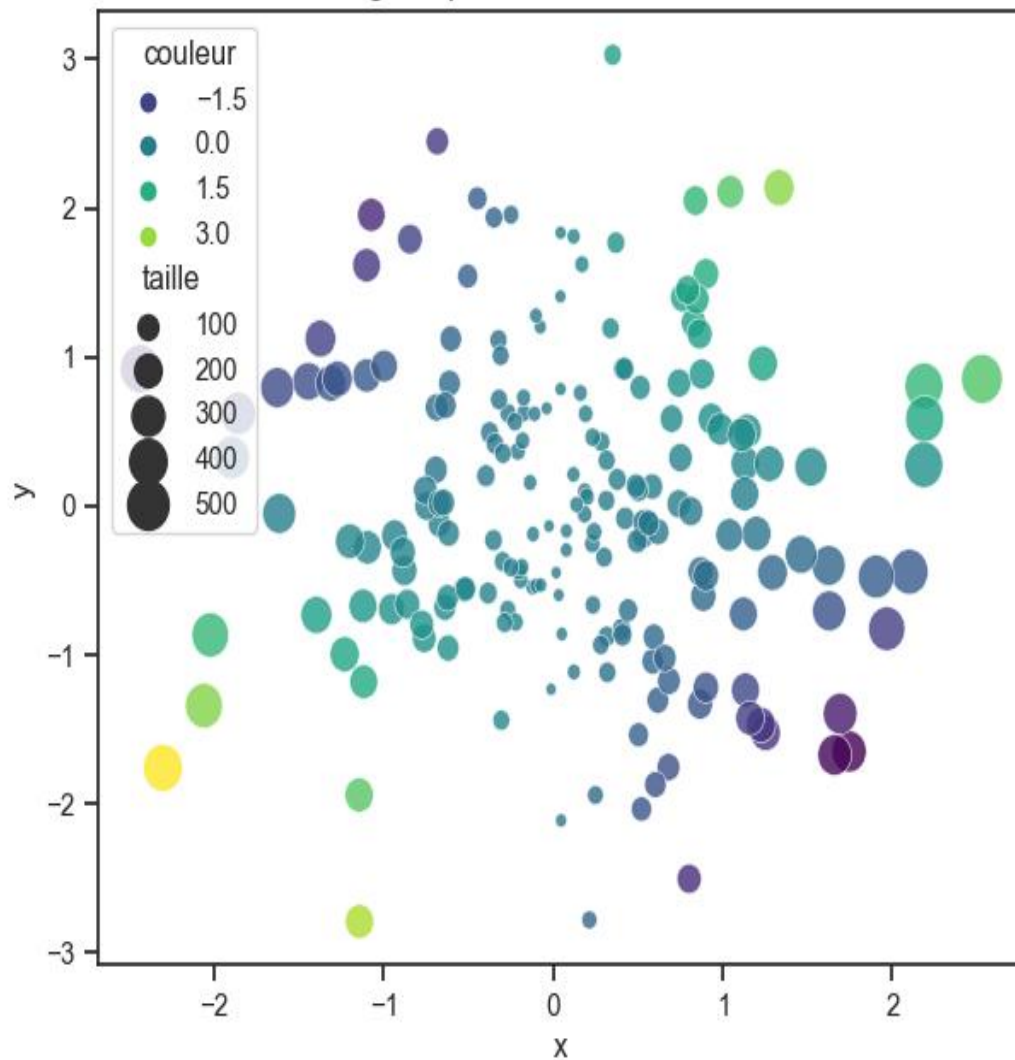
1 import matplotlib.pyplot as plt
2 import numpy as np
3 # Génération de données
4 np.random.seed(0)
5 x = np.random.randn(100); y = np.random.randn(100)
6 # Tracé du nuage de points
7 plt.figure(figsize=(6, 6))
8 plt.scatter(x, y, color="blue", alpha=0.7)
9
10 plt.title("Nuage de points avec Matplotlib")
11 plt.xlabel("X"); plt.ylabel("Y")
12 plt.grid(True) ; plt.show()

```



```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import numpy as np; import pandas as pd
4
5 np.random.seed(1)
6 x = np.random.randn(200); y = np.random.randn(200)
7 couleur = x * y
8 taille = np.abs(x) * 200  # agrandir les valeurs
9
10 df = pd.DataFrame({ "x": x,      "y": y,
11                     "couleur": couleur, "taille": taille })
12
13 sns.set_theme(style="ticks")
14 plt.figure(figsize=(7, 6))
15 |
16 sns.scatterplot( data=df, x="x", y="y", hue="couleur",
17                 size="taille", palette="viridis", alpha=0.8, sizes=(20, 300))
18 plt.title("Nuage de points avancé avec Seaborn"); plt.show()
```

Nuage de points avancé avec Seaborn



1 df

	x	y	couleur	taille
0	1.624345	-0.400878	-0.651165	324.869073
1	-0.611756	0.824006	-0.504091	122.351283
2	-0.528172	-0.562305	0.296994	105.634350
3	-1.072969	1.954878	-2.097523	214.593724
4	0.865408	-1.331952	-1.152681	173.081526
...	...	...	...	...
195	-1.627438	0.794528	-1.293046	325.487668
196	0.602319	-1.873161	-1.128241	120.463856
197	0.420282	0.920615	0.386918	84.056441
198	0.810952	-0.035368	-0.028682	162.190335
199	1.044442	2.110605	2.204405	208.888419

200 rows × 4 columns

Dash vs Streamlit : quelles différences ?

A- Dash: Développé par Plotly. Pensé pour créer des applications web professionnelles, robustes et personnalisables. Fonctionne comme un mini-framework web (avec des callbacks, une structure explicite, etc.). Convient bien pour des applications complexes, multi-pages, avec logique avancée.

B- Streamlit: Pensé pour être ultra-simple et rapide. Permet de transformer un script Python en interface web en quelques lignes. Très apprécié des data scientists pour le prototypage rapide, les démos, les dashboards légers. Streamlit est conçu par et pour les data scientists. Permet de transformer directement un notebook en interface utilisable.